

Laboratory Exercise

Writing Unit Tests Using White-Box Testing Approach and JUnit Tool

Objective:

At the end of the exercise, the students should be able to:

- Write unit tests using white-box and black-box testing approaches.

Materials:

- NetBeans IDE 8.2
- JDK 8
- 07 Software System for Testing folder
 - LoginModule.zip

Procedure:

1. Create a folder on your local drive with your name. (Ex. Jess_Diaz)
2. Open NetBeans IDE 8.2. Go to **File > New Project**. The **New Project** dialog box will appear. Select **Java** in **Categories > Java Application** in **Projects**. Then, click the **Next** button.
3. Write **TaxCalculator** in **Project Name** and check the **Use Dedicated Folder for Storing Libraries** option. Click **Finish** to create the project.
4. Go to **File > New File**. The **New File** dialog box will appear. Select **Java** in **Categories > Java Class** in **File Types**. Then, click the **Next** button.
5. Write **ItemTaxCalculator** in **Class Name** and select **taxcalculator** in **Package**. Click **Finish** to create the new class.
6. Write the following program in the **ItemTaxCalculator** class. The **CalculateItemTax** method will be used to calculate an item sales tax.

```
package taxcalculator;

public class ItemTaxCalculator {
    public double CalculateItemTax (double itemPrice, double taxPercent) {
        double decrease = taxPercent / 10.0;
        return itemPrice * decrease;
    }
}
```

7. Write a unit test using the JUnit unit testing tool available for NetBeans IDE. This unit test will be used to test the **CalculateItemTax** method.
8. Go to **File > New File**. The **New File** dialog box will appear. Select **Unit Tests** in **Categories > JUnit Test** in **File Types**. Then, click the **Next** button.

Note: If the JUnit Test is not installed in your IDE, go to **Tools > Plugins**. The **Plugins** dialog box will appear. Go to the **Available Plugins** tab and look for the **Search** textbox. Type **JUnit**. Check **JUnit** in the result area and click **Install**.

9. Write **ItemTaxCalculatorTest** in the **Class** name and write the package name **taxcalculator** in **Package**. Tick all the checkbox options in **Generated Code** and **Generated Comments**. Click **Finish** to create the unit test class and the **Test Packages** node.

Notes:

- When creating the unit test class, it is best to write the class name the same as the class (unit) to be tested and; word "Test" should be affixed to the class name.
 - The project requires a separate directory for the Test Packages to create tests. This directory will contain the unit tests for the project.
 - The package of class to be tested and the test class should be the same.
10. Go to the **Projects** directory to view the project. Your project directory for the current project should be the same as *Figure 1*.

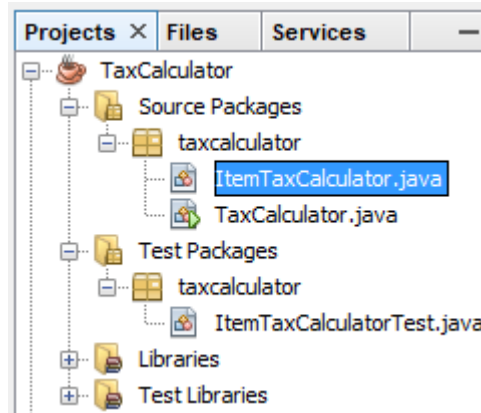


Figure 1. TaxCalculator project

11. To execute a test, go to **Projects**. Right-click the class file to test, which is the **ItemTaxCalculator**, then select **Test File**. This will run a test to the project but will result to Failed (see *Figure 2*). This is because there are no runnable test methods found in the test class.

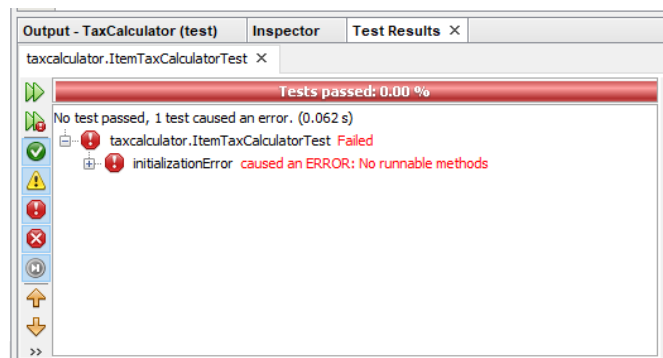


Figure 2. Test failed shown in Test Results window

12. To create a test method in the test class, write the following test method inside the **ItemTaxCalculatorTest** class:

```
// Test of CalculateItemTax method, of ItemTaxCalculator class

@Test
public void testCalculateItemTax() {
    System.out.println("CalculateItemTax method");
    ItemTaxCalculator instance = new ItemTaxCalculator ();
    double itemPrice = 50.0;
    double taxPercent = 1.0;
    double expectedResult = 5.0;
```

```
double result = instance.CalculateItemTax(itemPrice, taxPercent);
assertEquals(expectedResult, result, 0.0);
}
```

13. Run the Test File again to run the unit test. This should result to Passed (see Figure 3).

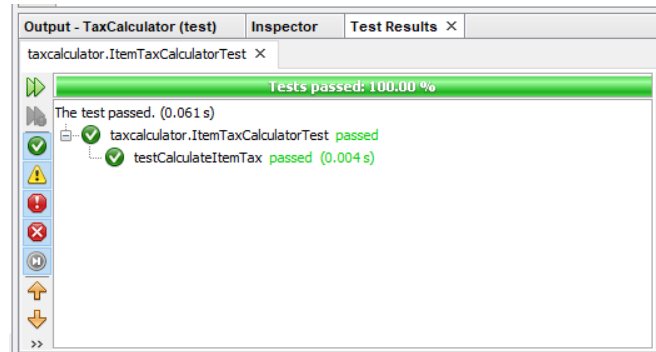


Figure 3. Test passed shown in Test Results window

- In the given test program, the method **CalculateItemTax()** is being tested.
- The test method **testCalculateItemTax()** contains the annotation **@Test**. This annotation specifies that method is the test method.
- Within the test method, the instance of **ItemTaxCalculator** class is created. This will be used to call the method to be tested.
- The test method contains test inputs, **itemPrice** and **taxPercent**, and the variable **expectedResult** that will hold the expected output of the test from the test data.
- The variable **result** will hold the actual result of the test and will be used to compare to the expected result. This variable will hold the result from the method of the created instance.
- The **assertEquals()** method belongs to the **org.junit.Assert** class. This method checks if the given expected result is equal to the actual result. This method is overloaded.

Note: If you run the program before executing the test, it is necessary to Clean and Build () the program before executing the test.

Activity:

1. Using NetBeans IDE, extract and open the java zipped project **LoginModule** in **07 Software System for Testing** folder. This project contains functions that allow users to log in to and create an account for the system. This also has functions that will compute basic salary based on hourly rate, hours worked, and overtime.
2. Create unit tests for the following classes of the system that will result all into "passed":
 - **VerifyUserLogin** (remove 0.0 from the assertEquals statement)
 - **PayrollCalculator**
3. Submit your output to your instructor. Your output will be graded based on the given rubric.

GRADING RUBRIC:

CRITERIA	PERFORMANCE INDICATORS	POINTS
Correctness	The code produces the expected result.	30
Logic	The code meets the specifications of the problem.	30
Efficiency	The code is concise without sacrificing correctness and logic.	20
Syntax	The code adheres to the rules of the programming language.	20
Total		100